

ARTIFICIAL INTELLIGENCE • BS COMPUTER SCIENCE •  
PRACTICAL / LAB 1

# AI Practical 1

Development Environment, Python & a Rule-Based  
Chatbot

---

Instructor: **Dr. Ghulam Ali**

*“Learn by doing — from your first line of Python to your first intelligent-looking program.”*

# What Will We Do Today?

A hands-on workflow — from environment setup to a working chatbot.

1

**Set up the Python environment**

2

**Explore Jupyter Notebook /  
Google Colab**

3

**Understand basic Python syntax**

4

**Write our first Python program**

5

**Use variables and user input**

6

**Use conditions and functions**

7

**Build a rule-based chatbot**

8

**Test and extend the chatbot**

9

**Understand why it appears “AI-like”**

# Learning Outcomes

By the end of this practical, you will be able to:

- ✓ Set up an AI development environment
- ✓ Create and execute a Python program
- ✓ Use variables, strings, input and output
- ✓ Write conditional statements

- ✓ Define and call functions
- ✓ Implement simple rule-based decision logic
- ✓ Build a basic rule-based chatbot
- ✓ Explain why rule-based systems can seem intelligent, and name their limitations

# Tools We Will Use

## Python

The programming language we use to express AI logic.

## Anaconda

A distribution that manages Python + AI libraries locally.

## Jupyter Notebook

An interactive, cell-based way to write and run code.

## Google Colab

A free, browser-based notebook — no installation needed.

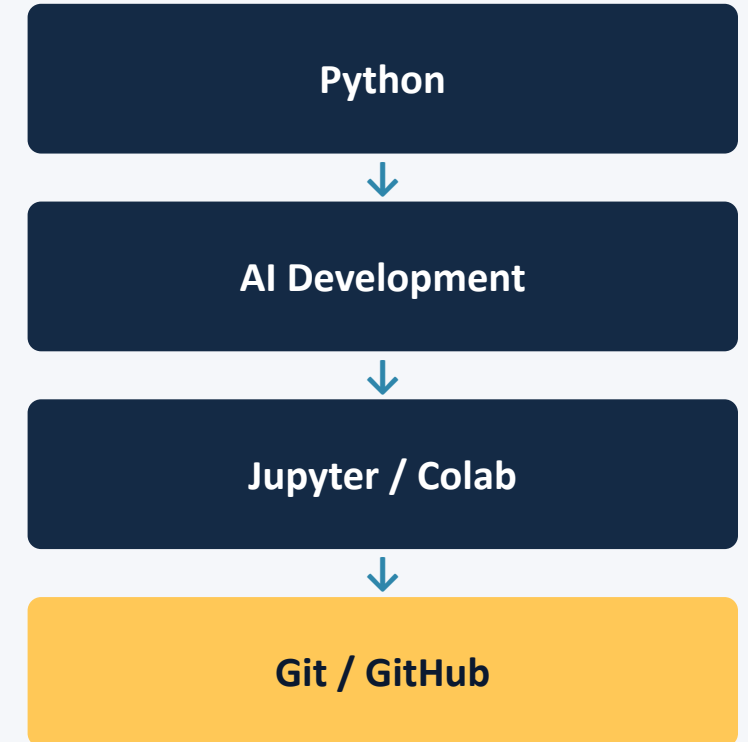
## Git

Tracks changes to your code over time.

## GitHub

Stores and shares your code online (brief intro only).

## HOW THEY CONNECT



# Why Do AI Students Need a Dev Environment?

- 1 **Writing code** a place to author Python programs
- 2 **Running experiments** executing and re-running AI logic quickly
- 3 **Installing libraries** adding tools like NumPy, Pandas, scikit-learn
- 4 **Managing notebooks** organizing work into shareable files
- 5 **Saving experiments** keeping results for later comparison
- 6 **Reproducing results** getting the same output again, reliably
- 7 **Version control** tracking how code changes over time

“

*AI is not only about algorithms;  
it also requires a reliable  
computational environment.*

# Option 1: Anaconda

- A free Python distribution built for data science and AI
- Bundles Python with hundreds of pre-installed packages
- Includes conda — a package and environment manager
- Lets you create isolated environments per project
- Installed once, used offline on your own machine

## LIBRARIES YOU WILL MEET LATER IN THIS COURSE

NumPy

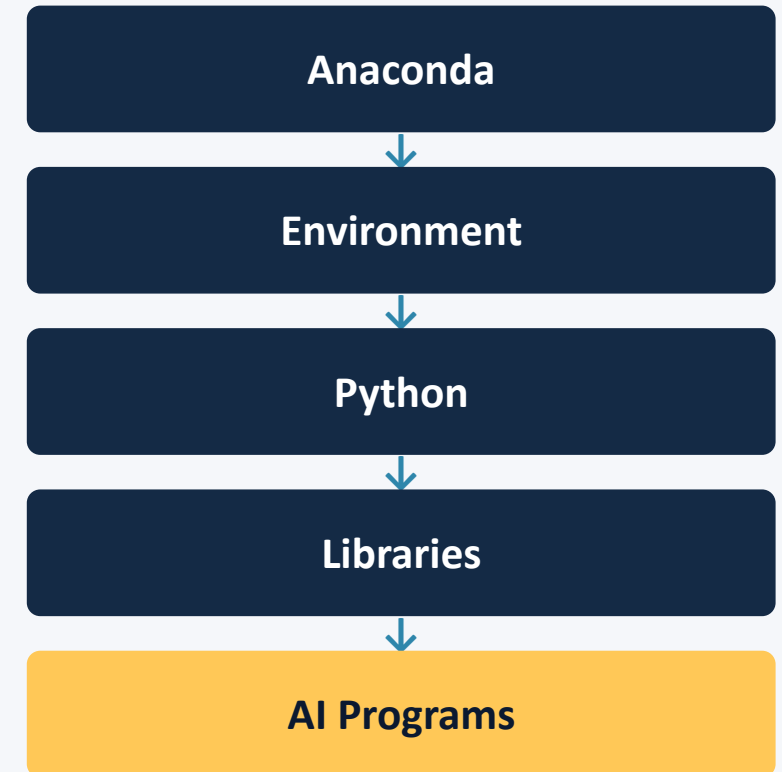
Pandas

Matplotlib

Scikit-learn

PyTorch / TensorFlow

## CONCEPTUAL WORKFLOW



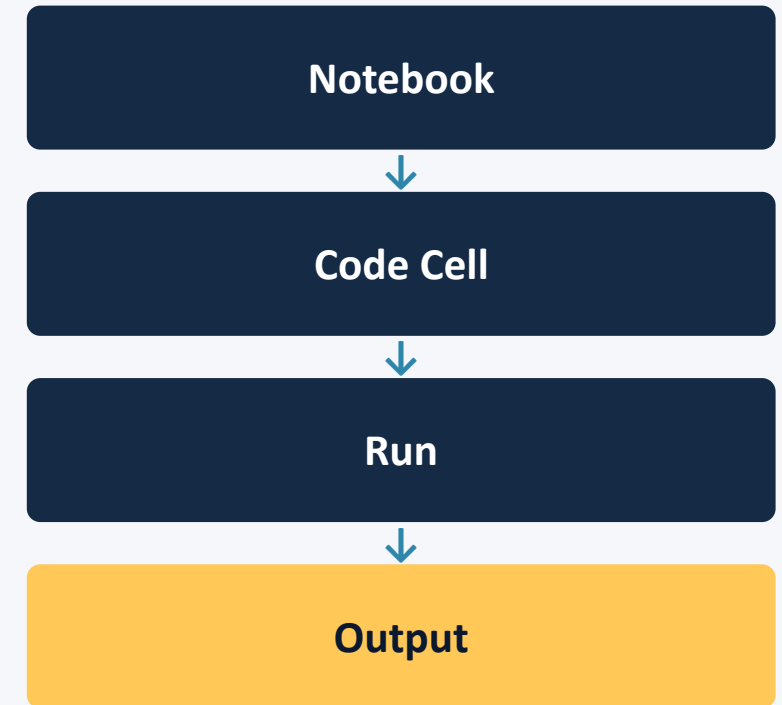
## Option 2: Google Colab

- A free, browser-based Python environment by Google
- No local installation required — runs in the cloud
- Jupyter Notebook-style interface (cells you can run individually)
- Great for quick AI experiments and sharing work
- Optional GPU access, useful in later AI/ML practicals

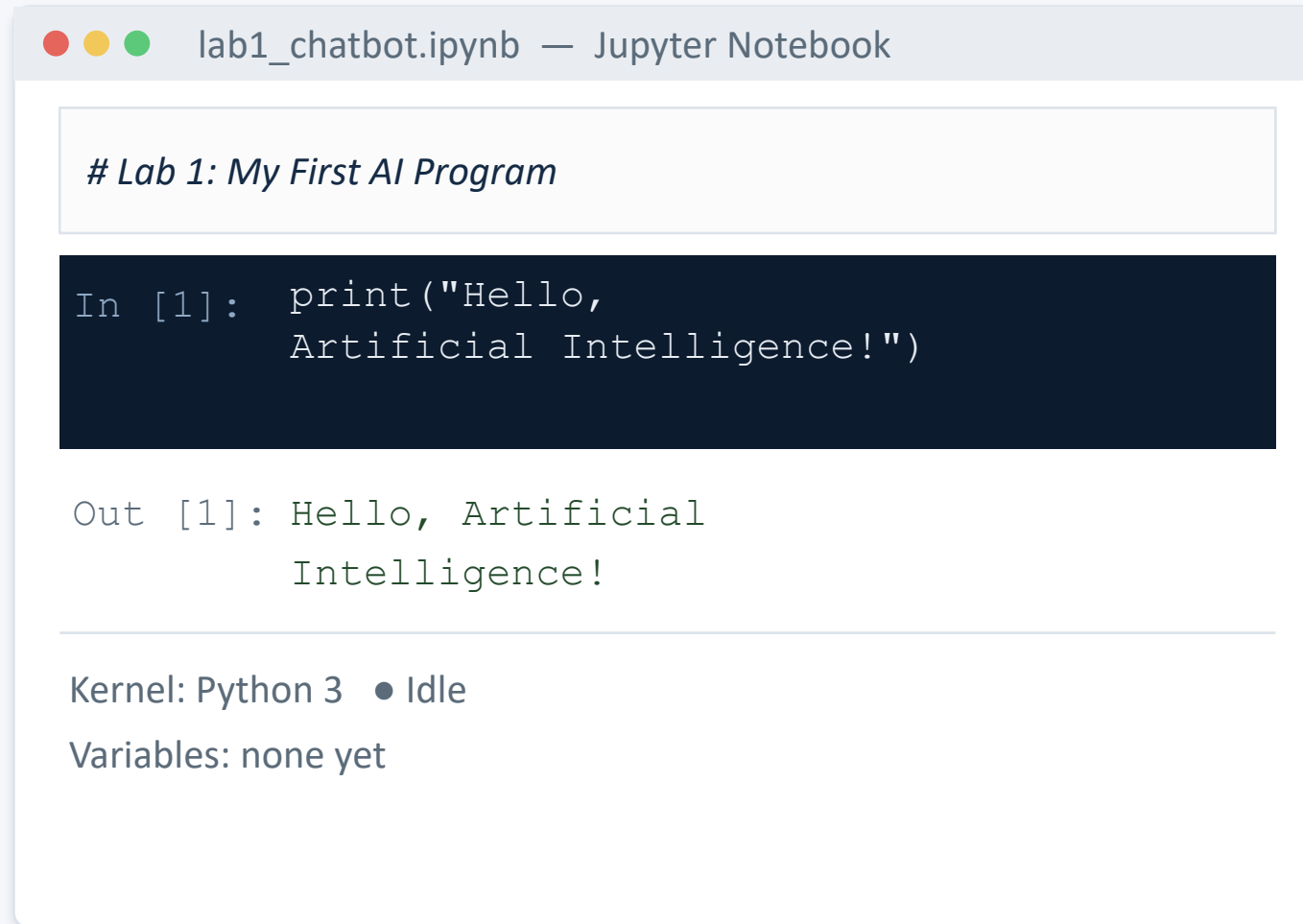
**Local Jupyter** runs on your machine, works offline, uses your own hardware.

**Google Colab** runs on Google's servers, needs internet, nothing to install.

### HOW A CELL RUNS



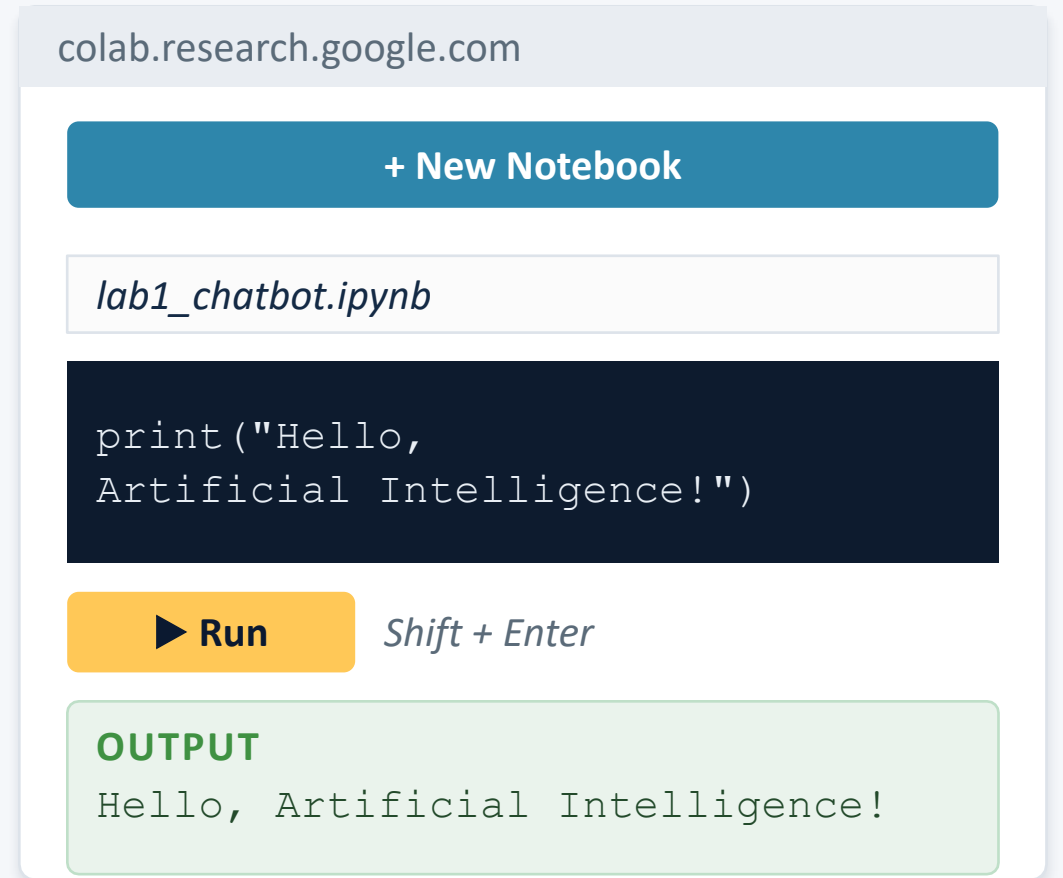
# Jupyter Notebook Interface



- 1 **Notebook** — the .ipynb file holding all your cells
- 2 **Code cell** — editable block where you write Python
- 3 **Markdown cell** — formatted text for notes and headings
- 4 **Run button / Shift+Enter** — executes the current cell
- 5 **Kernel / runtime** — the Python process running your code
- 6 **Output** — what the cell prints or returns

# Setting Up Google Colab — Step by Step

- 1 Open Colab** Go to `colab.research.google.com` in any browser.
- 2 Sign in** Log in with a Google account (create one if needed).
- 3 New notebook** Click “New Notebook” to get a blank `.ipynb` file.
- 4 Rename it** Click the title and rename to `lab1_chatbot.ipynb`.
- 5 Run your first cell** Type a print statement and press `Shift+Enter`.



# Anaconda vs. Google Colab — Which to Use?

Both are valid. This practical uses whichever your instructor selects for the lab.

| Feature          | Anaconda (local)                    | Google Colab (cloud)            |
|------------------|-------------------------------------|---------------------------------|
| Installation     | Required, one-time setup            | None — just open a browser      |
| Internet needed  | No, works offline                   | Yes, always online              |
| Hardware used    | Your own computer                   | Google's servers (GPU optional) |
| Best for         | Full control, larger local projects | Quick starts, easy sharing      |
| Setup time today | Longer                              | Under two minutes               |

**Recommendation for this course:** Google Colab is recommended for these practicals — it removes setup friction so we can focus on AI concepts. Anaconda remains a strong choice for your own projects outside class.

# Why Python for Artificial Intelligence?

- 1 Readable, simple syntax** close to plain English, easy to learn quickly
- 2 Huge AI/ML library ecosystem** NumPy, Pandas, scikit-learn, PyTorch, TensorFlow
- 3 Large community & documentation** answers and examples are easy to find
- 4 Used across industry and research** the de-facto language of modern AI
- 5 Fast to prototype in** test an idea in minutes, not hours

## PYTHON IN ONE LINE

```
print("AI starts here")
```

No compiling, no boilerplate — write one line and see a result immediately. That short feedback loop is why Python dominates AI experimentation.

# Python Syntax Basics

- Indentation defines blocks — no curly braces { }
- Comments start with # and are ignored when run
- Statements typically end with a new line, not a semicolon
- Python is case-sensitive: Name and name are different

## CORRECT

```
if age >= 18:
    print("Adult")
else:
    print("Minor")
```

## INCORRECT — INDENTATION ERROR

```
if age >= 18:
print("Adult")
# missing indentation before print
# → IndentationError
```

# Your First Python Program

Every programmer's tradition — make the computer say hello.

```
• • •  
# my_first_program.py  
print("Hello, Artificial Intelligence!")  
print("My AI journey starts today.")
```

## OUTPUT

```
Hello, Artificial Intelligence!  
My AI journey starts today.
```

- 1 **print()** a built-in function that displays text on screen
- 2 **Quotation marks** text (a string) needs " " or ' ' around it
- 3 **Run order** lines execute top to bottom, one at a time

# Variables and Data Types

```
name = "Alex"
age = 20
height = 5.9
is_student = True

print(type(name))
print(type(age))
```

| Type  | Example      | Meaning                       |
|-------|--------------|-------------------------------|
| str   | "Alex"       | text (a string of characters) |
| int   | 20           | a whole number                |
| float | 5.9          | a decimal number              |
| bool  | True / False | a logical yes/no value        |

*A variable is a labelled box that stores a value for later use.*

# Getting Input From the User

```
name = input("What is your name? ")
print("Hello, " + name + "!")

# input() always returns a string
age_text = input("Your age? ")
age = int(age_text)      # convert to a
number
```

## OUTPUT

```
What is your name? Sara
Hello, Sara!
Your age? 21
```

## KEY IDEA

**input()** always returns text (a string)  
— even if the user types a number.

Use **int(...)** or **float(...)**  
to convert it before doing math with it.

This is exactly what our chatbot will use to  
read what the user types.

# String Formatting and Output

```
name = "Sara"
age = 21

# f-string: the modern, readable way
print(f"Hello, {name}! You are
      {age} years old.")
```

## OUTPUT

```
Hello, Sara! You are 21 years old.
```

*Why this matters for our chatbot: every reply the chatbot gives is built with f-strings — mixing fixed text with variables like the user's name.*

## F-STRING SYNTAX

**f" ...{variable} ... "**

- Put an `f` directly before the opening quote
- Wrap variable names in curly braces `{ }`
- Python fills in the value automatically
- Much cleaner than joining strings with `+`

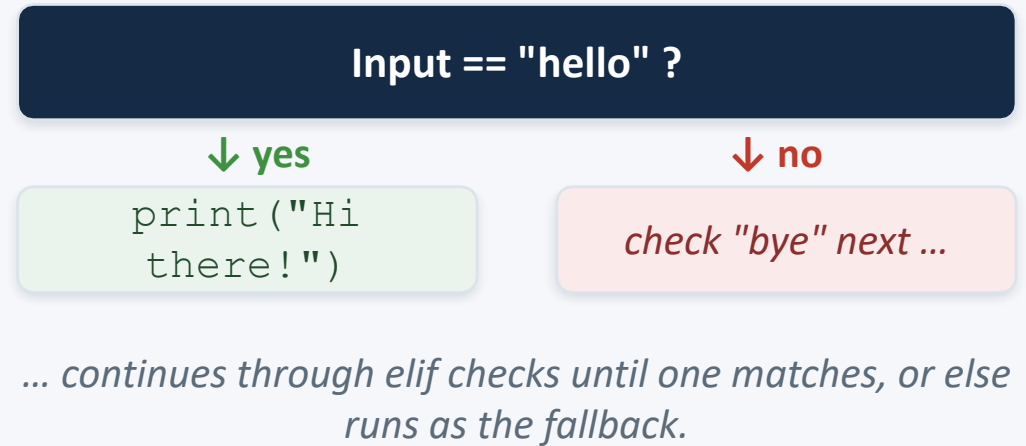
# Conditional Statements: if / elif / else

```
user_input = "hello"

if user_input == "hello":
    print("Hi there!")
elif user_input == "bye":
    print("Goodbye!")
else:
    print("I did not understand that.")
```

*This exact pattern — checking input against known cases — is the foundation of our rule-based chatbot.*

## DECISION FLOW



**Comparison operators:** `==` equals, `!=` not equal, `in` membership test (e.g. `"hi" in user_input`)

# Functions: Organizing Your Code

```
def greet(user_name):  
    message = f"Hello, {user_name}!  
                How can I help?"  
    return message  
  
reply = greet("Sara")  
print(reply)
```

## OUTPUT

```
Hello, Sara! How can I help?
```

- 1 **def** — keyword that starts a function definition
- 2 **Parameters** — values passed into the function, e.g. `user_name`
- 3 **return** — sends a value back to whoever called the function
- 4 **Reusable** — call it many times with different inputs

# Lists: Storing Multiple Values

```
greetings = ["hello", "hi", "hey"]
user_input = "hey there"

for word in greetings:
    if word in user_input:
        print("Greeting recognised!")
        break
```

## OUTPUT

```
Greeting recognised!
```

## WHY LISTS MATTER HERE

A **list** groups related keywords together, e.g. all the ways a user might say hello.

The **in operator** checks membership — “is this word inside that text?”

Our chatbot will use a list of keywords for every kind of response it can give.

# Practice: Build a Mini Decision Program

Ten minutes, on your own or in pairs — apply everything from this section.

## YOUR TASK

1. Ask the user for their name using `input()`.
2. Ask the user to enter a time of day: morning, afternoon, or evening.
3. Use `if / elif / else` to print a different greeting for each case, including the user's name.
4. Wrap your greeting logic inside a function called `greet_by_time()`.

```
def greet_by_time(name, tod):  
    if tod == "morning":  
        return f"Morning, {name}!"  
    elif tod == "afternoon":  
        return "____" # fill in  
    elif tod == "evening":  
        return "____" # fill in  
    else:  
        return "____" # fill in
```

*Hint: reuse the exact if/elif/else pattern from the previous slides — only change the conditions and messages.*

# What Is Rule-Based Decision Making?

A rule is simply: IF a condition is true, THEN take a specific action.

| If the input contains... | Then respond with...                     |
|--------------------------|------------------------------------------|
| "hello", "hi", "hey"     | A greeting back to the user              |
| "bye", "goodbye"         | A farewell message, then stop            |
| "name"                   | "I am a rule-based chatbot!"             |
| "weather"                | "I cannot check the weather yet."        |
| none of the above        | A fallback: "I did not understand that." |

## THE CORE IDEA

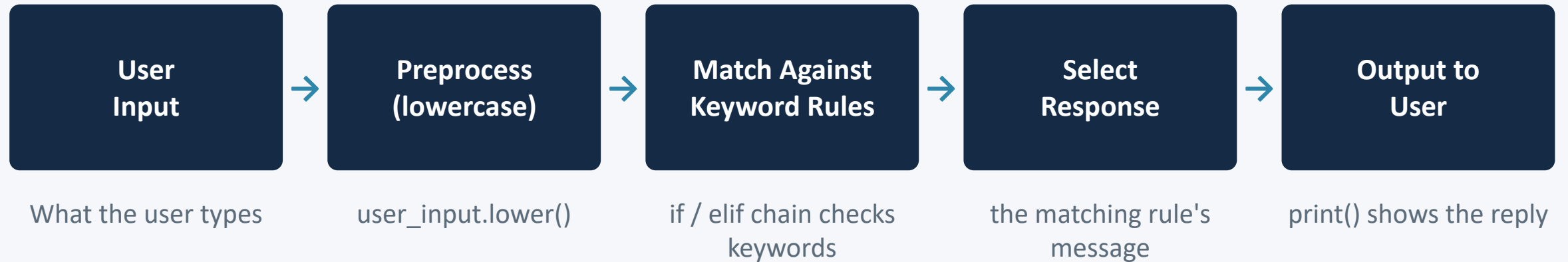
A rule-based system does not “understand” language.

It matches patterns against a fixed list of rules written in advance by a programmer, and picks the response tied to the first rule that matches.

**This is one of the earliest and simplest forms of “AI.”**

# Designing the Chatbot's Logic

Every chatbot turn follows the same pipeline, no matter how simple or complex.



## EXAMPLE WALK-THROUGH

**User types:** "Hey, what is your NAME?" → lowercased to "hey, what is your name?"

**Rule check:** "name" in `user_input` → True, so this rule matches.

**Response:** "I am a rule-based chatbot!"

# Step 1: Recognising a Greeting

```
def chatbot_response(user_input):  
    user_input = user_input.lower()  
    if "hello" in user_input or "hi" in user_input:  
        return "Hi there! How can I help you  
today?"  
    else:  
        return "I did not understand that."
```

## OUTPUT

```
chatbot_response("Hi there!")  
→ "Hi there! How can I help you today?"
```

- 1 **Single responsibility** one function handles one reply per call
- 2 **or** matches if either keyword is present
- 3 **Fallback first** start simple, add more rules next

## Step 2: Expanding the Rule Set

```
def chatbot_response(user_input):  
    user_input = user_input.lower()  
    if "hello" in user_input or "hi" in user_input:  
        return "Hi there! How can I help?"  
    elif "your name" in user_input:  
        return "I am a rule-based chatbot!"  
    elif "how are you" in user_input:  
        return "Just a program, thanks for asking!"  
    elif "bye" in user_input:  
        return "Goodbye! Have a great day."  
    elif "thank" in user_input:  
        return "You are welcome!"  
    else:  
        return "Sorry, I did not understand."
```

### DESIGN NOTE

**Each `elif`** is one more rule the chatbot knows.

**Order matters:** the first matching rule wins, even if a later rule would also match.

**The `else` fallback** always stays last — it catches everything else.

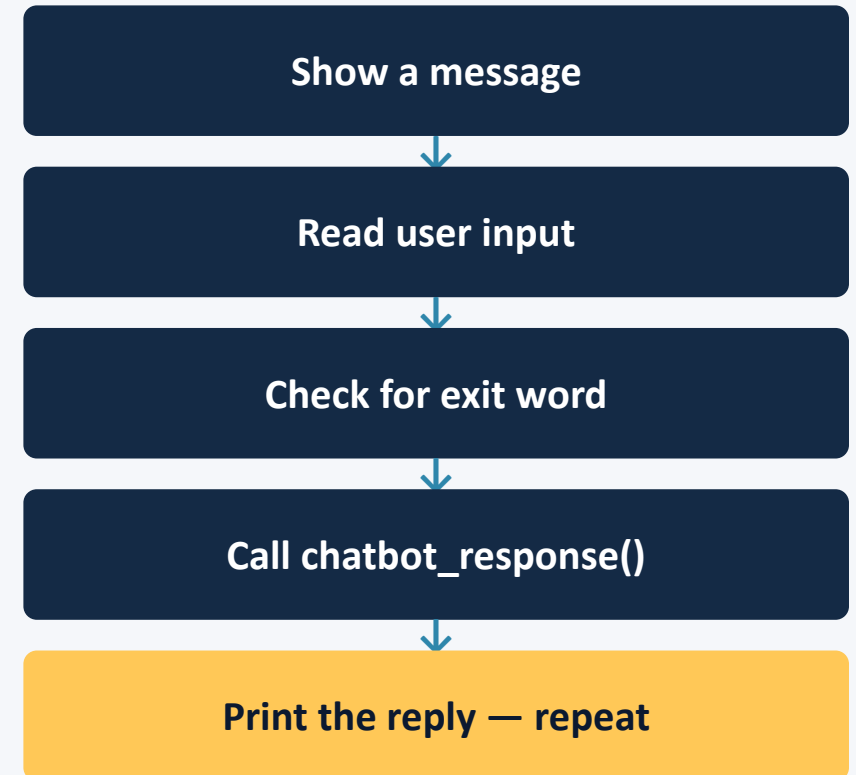
## Step 3: The Conversation Loop

```
print("Chatbot: Hi! Type 'bye' anytime to exit.")
while True:
    user_input = input("You: ")
    if "bye" in user_input.lower():
        print("Chatbot: Bye! Have a great day.")
        break
    response = chatbot_response(user_input)
    print("Chatbot:", response)
```

### OUTPUT

```
You: hello
Chatbot: Hi there! How can I help?
You: bye
Chatbot: Bye! Have a great day.
```

### WHY A while LOOP



# The Complete Program

Everything from Steps 1–3, combined into one runnable file.

```
def chatbot_response(user_input):
    user_input = user_input.lower()
    if "hello" in user_input or "hi" in
user_input:
        return "Hi there! How can I help?"
    elif "your name" in user_input:
        return "I am a rule-based
chatbot!"
    elif "how are you" in user_input:
        return "Just a program, thanks for
asking!"
    elif "thank" in user_input:
        return "You are welcome!"
    else:
        return "Sorry, I did not
understand."
```

```
# ...continued: run the conversation

print("Chatbot: Hi! Type 'bye' to exit.")
while True:
    user_input = input("You: ")
    if "bye" in user_input.lower():
        print("Chatbot: Bye!")
        break
    print("Chatbot:",
          chatbot_response(user_input))
```

# Testing and Extending Your Chatbot

## TEST THESE INPUTS

- ? "Hello!"
- ? "What is your name?"
- ? "How are you doing?"
- ? "asdkjfh random text"
- ? "thank you so much"
- ? "bye"

## NOW EXTEND IT

1. Add three new elif rules of your own (e.g. "joke", "help", "age").
2. Ask for the user's name once at the start, and personalise every reply with it using an f-string.
3. Try an input that matches none of your rules — confirm the fallback still works.
4. Save your finished file — you will build on it in a future practical.

# Why Does This Chatbot Seem “AI-like”?

It feels intelligent because of what we, as humans, project onto it — not because it understands.

## WHAT THE USER PERCEIVES

*“It answered my question directly.”*

*“It responds differently to different things I say.”*

*“It feels like a real back-and-forth conversation.”*

*“It seems to know things about itself.”*

## WHAT IS ACTUALLY HAPPENING

A keyword happened to be in a pre-written list

Different keywords trigger different elif branches

The while loop simply repeats the same single-turn check

A hardcoded string was returned for that one rule

# Limitations of Rule-Based AI

| Capability                 | Rule-Based Chatbot (today)                        | Modern AI / ML Systems                    |
|----------------------------|---------------------------------------------------|-------------------------------------------|
| Handling unseen input      | Fails silently — falls back to a generic reply    | Generalises from patterns learned in data |
| Learning over time         | Never improves; rules are fixed by the programmer | Can improve as more data is seen          |
| Understanding context      | No memory of earlier turns in the conversation    | Can track context across a conversation   |
| Handling ambiguity / typos | Breaks on misspellings or unexpected phrasing     | More robust to variation and noise        |
| Scalability                | Every new topic needs a hand-written rule         | Learns broad behaviour from examples      |

**Takeaway:** Rule-based systems are a useful starting point and are still used where behaviour must be fully predictable — but they cannot learn or adapt. This gap is exactly what the rest of this course (search, knowledge representation, and machine learning) will address.

## PRACTICAL 1 — WRAP-UP

# Summary & What's Next

### TODAY YOU:

- ✓ Set up a Python environment (Anaconda / Colab)
- ✓ Learned core Python: variables, input, conditions, functions, lists
- ✓ Designed and coded a rule-based chatbot from scratch
- ✓ Explained why it feels “AI-like”, and where it breaks down

### COMING NEXT IN THIS COURSE:

- Problem solving as search
- Knowledge representation
- Introduction to machine learning
- Natural language processing

### LAB TASK / HOMEWORK

1. Finish extending your chatbot with at least 3 new rules.
2. Personalise it with the user's name using an f-string.
3. Write 3–4 sentences (as a comment) on one limitation you personally observed.
4. Submit your .py or .ipynb file before the next practical.